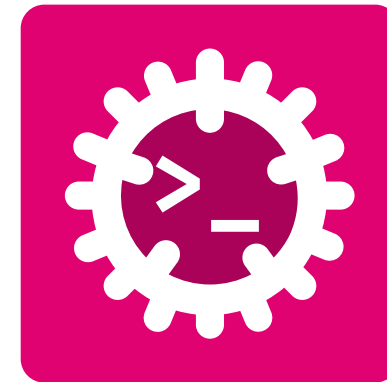




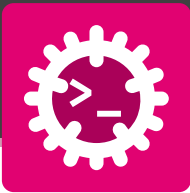
Advanced Programming

Design Patterns

Silvan Zahno

University of Applied Sciences Western Switzerland - HES-SO Valais Wallis
Systems Engineering
Infotronics

2026-09-10 - v0.1.0



Contents

Design Patterns 3

 What is a Design Pattern? 4

 Standard Library Traits 5

 Standard Library Traits 6

 Traits who can be derived 7

 Operators Are traits 8

 fmt:Display trait 9

 Default trait 10

 Clone trait vs Copy trait 11

 Comparison traits PartialEq, Eq, PartialOrd, Ord & Hash trait 12

 Conversion traits From / Into trait 13

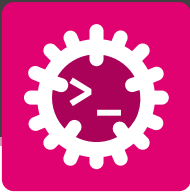
 Drop trait: Automatic Cleanup 14

 Iterator trait 16

 Exercise - Standard Library Traits 17

 Exercise - Standard Library Traits 18





Idiomatic Patterns 19

Idiomatic Patterns 20

Newtype Pattern 21

[Resource Acquisition Is Initialization \(RAII\)](#) / Drop 22

Singleton Pattern 23

Builder Pattern 24

Strategy Pattern 25

Exercise - Idiomatic Patterns 26

Exercise - Idiomatic Patterns 27

Glossary & Bibliography 28



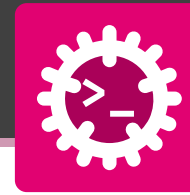


Design Patterns

Solutions to problems you'll eventually create

Design Patterns

—



A **design pattern** is a named, reusable solution to a problem that keeps recurring

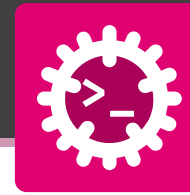
- **Classic Object-Oriented Programming (OOP)** expresses them as arrangements of classes (the “Gang of Four” catalogue)
- **Rust** often bakes them into the **type system** and **standard traits** - less boilerplate, checked by the compiler
- This chapter covers two families:
 - **Standard-library traits** - implement them and your type plugs into the language
 - **Idiomatic patterns** - the shapes that feel native in Rust

Design Patterns — What is a Design Pattern?



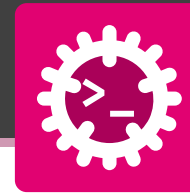
Standard Library Traits

Implement these and your type speaks Rust



Rust has **no class inheritance** - types share behaviour by **implementing traits**, never by extending a base class (polymorphism comes from `dyn Trait` / generics). Implement these standard traits to plug your type into the language - most are derivable with `#[derive(...)]`.

- **Formatting** - `Debug` (`{:?}`), `Display` (`{}`)
- **Duplication** - `Clone` (explicit), `Copy` (implicit, bitwise)
- **Defaults** - `Default` $\Rightarrow$ `T::default()`
- **Equality & ordering** - `PartialEq`/`Eq`, `PartialOrd`/`Ord`, `Hash`
- **Conversion** - `From`/`Into`, `TryFrom`/`TryInto`, `AsRef`/`AsMut`
- **Iteration** - `Iterator` (`map` / `filter` / `collect`), `IntoIterator`
- **Resource cleanup** - `Drop`
- **Operators** - `std::ops` (`Add`, `Mul`, `Index`, ...)



For the common traits, the compiler writes the implementation for you with `#[derive(...)]`.

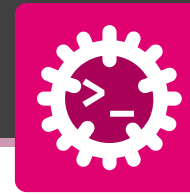
- One line generates correct, idiomatic implementations

```
#[derive(Debug, Clone, PartialEq, Default)]
struct Point { x: i32, y: i32 }

fn main() {
    let a = Point::default(); // Default → { x: 0, y: 0 }
    let b = a.clone();        // Clone
    println!("{a:?}");         // Debug → Point { x: 0, y: 0 }
    println!("{}", a == b);    // PartialEq → true
}
```

```
#[derive(Debug, Clone, PartialEq, Default, ...)]
```

- Derivable when **every field** of a struct also implements the **trait**
- Hand-write the `impl` only when you need custom behaviour
- Same mechanism you've seen on `Debug`, `Clone`, `Default`



Overloading an operator means **implementing the matching trait** from `std::ops`.

```
use std::ops::Add;
use std::fmt;

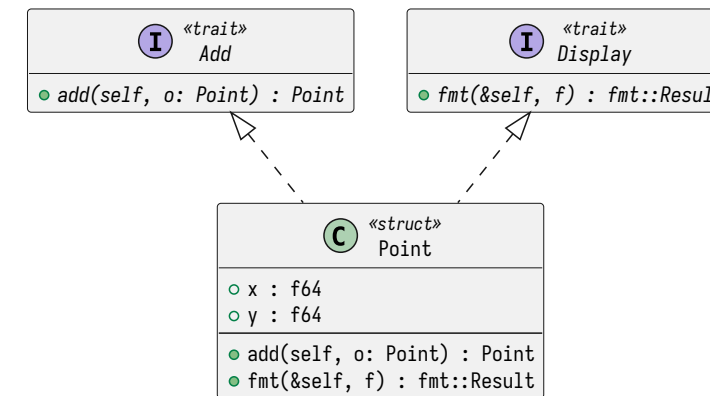
#[derive(Clone, Copy)]
struct Point { x: f64, y: f64 }

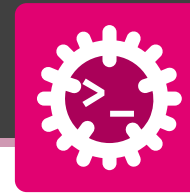
impl Add for Point {           // enables a + b
    type Output = Point;
    fn add(self, pt: Point) → Point {
        Point { x: self.x + pt.x, y: self.y + pt.y }
    }
}

impl fmt::Display for Point {  // enables {}
    fn fmt(&self, f: &mut fmt::Formatter) → fmt::Result {
        write!(f, "{{}, {}}", self.x, self.y)
    }
}

fn main() {
    let pt = Point { x: 1.0, y: 2.0 } + Point { x: 3.0, y: 4.0 };
    println!("{pt}");          // (4, 6)
}
```

- `+` $\Rightarrow$ Add, `*` $\Rightarrow$ Mul, `[]` $\Rightarrow$ Index, ...
- `Display` is used in String formatting as `{}`
- `Debug` is used in String formatting as `{:?}`
- An operator is **just a method call** on a trait
- Operator behaviour is **opt-in** and type-checked





Display (the {} formatter) is **not derivable** - you decide how your type reads to a human.

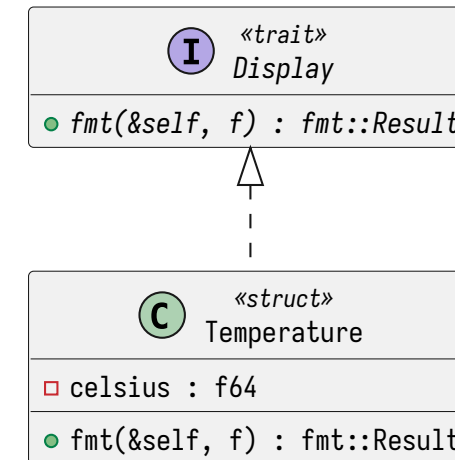
- Implement the `fmt::Display` trait.
- `write!(f, ...)` returns `fmt::Result`
- Implementing Display gives you `.to_string()` **for free**

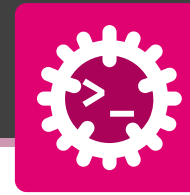
```
use std::fmt;

struct Temperature { celsius: f64 }

impl fmt::Display for Temperature {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        write!(f, "{:.1} °C", self.celsius) // your format
    }
}

fn main() {
    let t = Temperature { celsius: 21.5 };
    println!("{}", t); // 21.5 °C (uses Display)
    let s = t.to_string(); // "21.5 °C" (ToString is free)
}
```

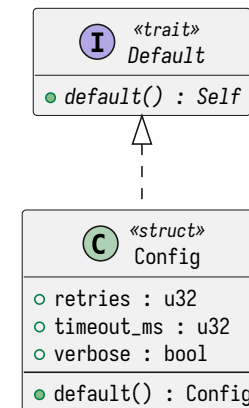




`#[derive(Default)]` fills every field with **its** default (`0`, `false`, `""`). Implement `Default` by hand when you need **different** default values.

```
struct Config {
    retries: u32,
    timeout_ms: u32,
    verbose: bool,
}
impl Default for Config {
    fn default() → Self {
        Self {
            retries: 3,      // not 0
            timeout_ms: 5000, // not 0
        }
    }
}
fn main() {
    let c = Config::default(); // retries: 3, timeout: 5000
    let c2 = Config {
        retries: 5, // override one field
        ..Default::default() // fill the rest with defaults
    };
}
```

- Derived default = **all-zero / empty**; sometimes that's wrong
- Implement `fn default() → Self` to pick **meaningful** defaults
- `..Default::default()` still works to fill the untouched fields





Both duplicate a value - the difference is **cost** and **when** it happens. Copy requires Clone.

```
#[derive(Clone, Copy)]           // Copy requires Clone
struct Point { x: i32, y: i32 }  // cheap, on the stack

#[derive(Clone)]                 // String owns a heap buffer → no Copy
struct Name { value: String }

fn main() {
    let p = Point { x: 1, y: 2 };
    let q = p;                    // COPIED implicitly
    println!("{}", p.x);         // ✓ p still usable

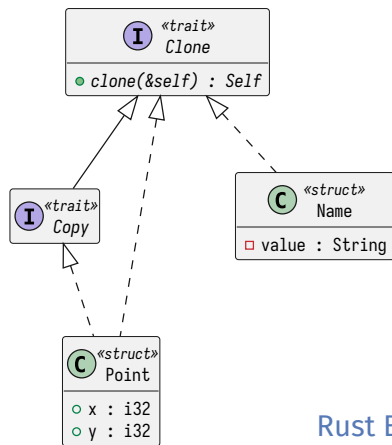
    let n = Name { value: "Ada".into() };
    let m = n.clone();            // CLONED explicitly (deep copy)
    println!("{}", m);           // ✓ both still usable
    println!("{}", n);           // ✓

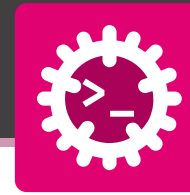
    let m = n;                    // would MOVE n, not copy it
    println!("{}", m);           // ✓ n was moved in to m
    println!("{}", n);           // ✗ n is no longer valid
}
```

- **Copy** - **implicit**, bit-for-bit duplicate on assignment; the original stays valid
 - only for stack-only types (ixx, fxx, bool, ...)
- **Clone** - **explicit** .clone()
 - expensive (deep copy)



If Copy does not exist ⇒ assignment = **moves** instead of copies.





Derive these to compare, sort, and key your type.

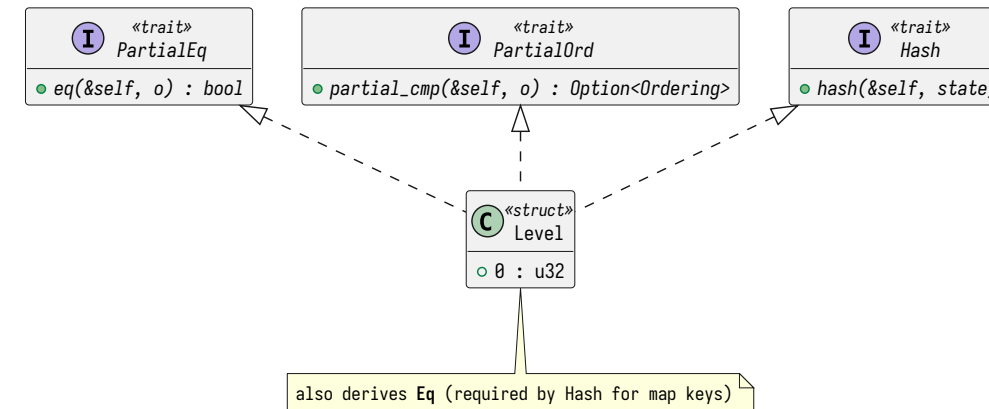
```
use std::collections::HashMap;
#[derive(PartialEq, PartialOrd, Eq, Hash)]
struct Level(u32);

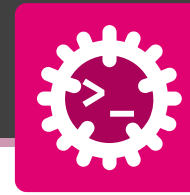
fn main() {
    println!("{}", Level(1) == Level(1)); // PartialEq → true
    println!("{}", Level(1) < Level(2)); // PartialOrd → true

    let mut v = vec![Level(3), Level(1), Level(2)];
    v.sort(); // ordering → [1, 2, 3]

    let mut seen = HashMap::new(); // Hash → usable as a key
    seen.insert(Level(1), "low");
    println!("{}", seen.get(&Level(1))); // Some("low")
}
```

- **PartialEq** $\Rightarrow$ defines $=$ and $\neq$
 - **PartialOrd** $\Rightarrow$ defines $< > \leq \geq$.sort()
 - **Hash** $\Rightarrow$ reduces a value to a number so a HashMap / HashSet knows which bucket to use
 - For a hash: $a == b \Rightarrow \text{hash}(a) == \text{hash}(b)$
so Hash and equality Eq must agree
- derive them together**





Implement From and you get Into **for free** - the idiomatic way to convert between types.

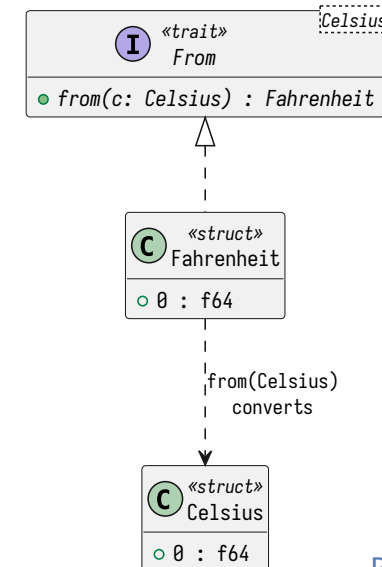
- Implement From<A> for B $\Rightarrow$ a.into() works automatically
- **Fallible** conversion $\Rightarrow$ TryFrom / TryInto (returns Result)
- The ? operator uses From to convert error types
- Convention: implement From, never Into directly

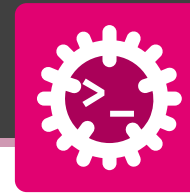
```
struct Celsius(f64);
struct Fahrenheit(f64);

impl From<Celsius> for Fahrenheit {
    fn from(c: Celsius) -> Self {
        Fahrenheit(c.0 * 9.0 / 5.0 + 32.0)
    }
}

fn main() {
    // call From directly - explicit about the target type
    let a = Fahrenheit::from(Celsius(100.0));
    println!("{}", a.0);           // 212

    // .into() - same conversion, target inferred from the type
    let b: Fahrenheit = Celsius(0.0).into();
    println!("{}", b.0);           // 32
}
```





The Drop trait runs code when a value goes out of scope - Rust's destructor, the basis of C++ [RAII](#). Same object, three approaches:

 **Rust** - Drop, automatic:

```
struct Widget { id: u32 }
impl Drop for Widget {
    fn drop(&mut self) {
        println!("drop {}", self.id);
    }
}

fn main() {
    let _w = Widget { id: 1 };
    println!("using...");
} // _w dropped here
// → "drop 1"
```

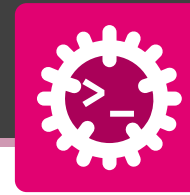
 **C++ RAI** - destructor, automatic:  **C++ classic** - manual delete:

```
struct Widget {
    int id;
    ~Widget() {
        std::cout << "drop " << id << "\n";
    }
};

int main() {
    Widget w{1};
    std::cout << "using...\n";
} // w destroyed here
// → "drop 1"
```

```
struct Widget {
    int id;
    ~Widget() {
        std::cout << "drop " << id << "\n";
    }
};

int main() {
    Widget* w = new Widget{1};
    std::cout << "using...\n";
    delete w; // × forget = leak
} // → "drop 1"
```



- **Rust & C++ RAI** - cleanup is **automatic** at end of scope, on **every** path
 - runs in reverse order of creation
 - no garbage collector
- **C++ classic** - **you** must delete
 - a missed branch **leaks**
 - a double delete is **undefined behaviour**
 - use-after-delete is a **dangling pointer**



Rust goes further - Drop is automatic **and** ownership makes double-free impossible; drop takes `&mut self` and can't be called by hand (drop early with `std::mem::drop(x)`)

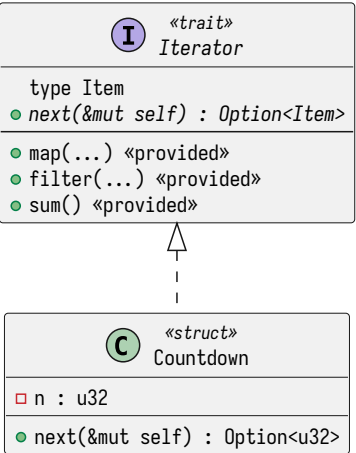


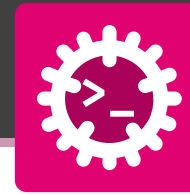
Implement one method - next() - and the whole iterator toolbox (map, filter, sum, collect, ...) comes for free.

- Define type Item and next() returning Option<Item>
- None signals the end of iteration
- Every adapter (map, filter, take, ...) builds on next()
- Lazy: nothing runs until consumed by sum / collect / a loop

```
struct Countdown { n: u32 }
impl Iterator for Countdown {
    type Item = u32;
    fn next(&mut self) -> Option<u32> {
        if self.n == 0 { return None; } // None = done
        self.n -= 1;
        Some(self.n + 1)
    }
}

fn main() {
    let total: u32 = Countdown { n: 3 } // yields 3, 2, 1
        .map(|x| x * 2) // 6, 4, 2
        .sum();
    println!("{total}"); // 12
}
```



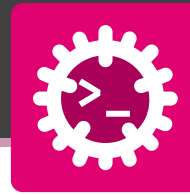


The program uses **four** standard traits, but the struct derives **none** of them. Add the **single** `#[derive(...)]` line that makes it compile.

```
struct Coord { x: i32, y: i32 }  
  
fn main() {  
    let a = Coord::default(); // ?  
    let b = a.clone();        // ?  
    println!("{a:?}");        // ?  
    println!("{}", a == b);   // ?  
}
```

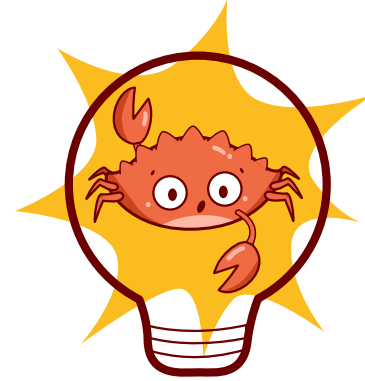


Each call names one trait: `default()`, `.clone()`, `{:?}`, `==`.

**Solution**

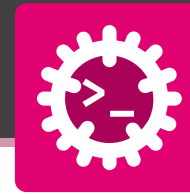
```
#[derive(Default, Clone, Debug, PartialEq)]
struct Coord { x: i32, y: i32 }

fn main() {
    let a = Coord::default(); // Default    → Coord { x: 0, y: 0 }
    let b = a.clone();        // Clone
    println!("{a:?}");        // Debug      → Coord { x: 0, y: 0 }
    println!("{}", a == b);   // PartialEq → true
}
```



Idiomatic Patterns

Patterns that feel native in Rust

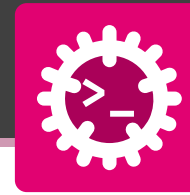


Patterns that fall naturally out of Rust's own features - not workarounds borrowed from class-based languages.

- The language often **is** the pattern: implement a std trait (Iterator, Drop) and you're done
- Others are conventions the type system rewards (Newtype, Builder)

Jump to any pattern:

- **Newtype** - a type-safe wrapper
- **RAII / Drop** - acquire in new(), release automatically in Drop
- **Singleton** - one shared instance via OnceLock<T> / static
- **Builder** - step-by-step construction of complex structs
- **Strategy** - interchangeable behaviour via trait objects or generics



```

struct Meters(f64);
struct Seconds(f64);
impl Meters {
    fn value(&self) → f64 { self.0 }
    // distance + distance = distance
    fn add(&self, other: &Meters) → Meters {
        Meters(self.0 + other.0)
    }
}
impl Seconds { fn value(&self) → f64 { self.0 } }
fn speed(d: &Meters, t: &Seconds) → f64 { // calculate speed
    d.0 / t.0
}
fn main() {
    let d = Meters(100.0);
    let total = d.add(&Meters(5.0)); // Meters(105.0)

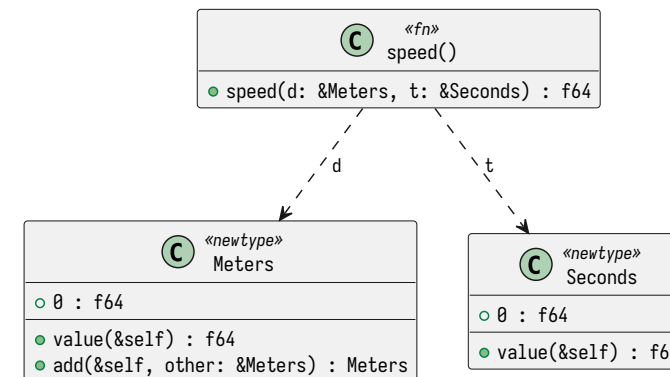
    let v = speed(&total, &Seconds(9.58)); // uses both types
    println!("{}", total.value()); // 105
    println!("{:.2}", v); // 10.97

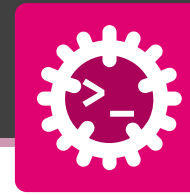
    speed(&total, &Seconds(9.58),); // ✓ compiles - Meters, Seconds
    speed(&total, 9.87); // ✗ won't compile - Meters, f64
    speed(&Seconds(9.58), &total); // ✗ won't compile - Seconds, Meters
}

```

Wrap a value in a one-field struct to give it a **distinct type**

- A **tuple struct** with one field - zero runtime cost
- The compiler intervenes on misuse
- Meters and Seconds are **different types**, even though both hold an f64
- Encodes **units / intent** into the type system

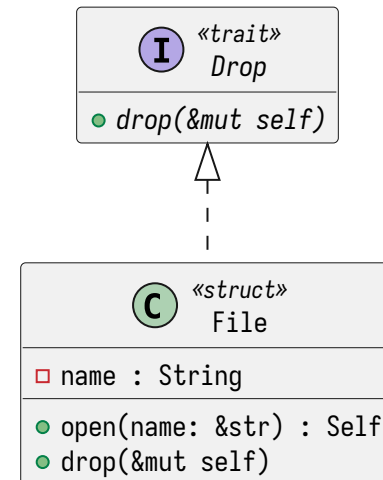


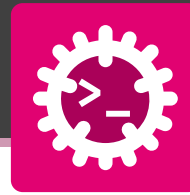


Tie a resource's lifetime to a value: acquire it in the constructor, release it automatically in Drop.

```
struct File { name: String }  
impl File {  
    fn open(name: &str) → Self {  
        println!("opening {name}");  
        Self { name: name.into() }  
    }  
}  
  
impl Drop for File {  
    fn drop(&mut self) { // runs automatically  
        println!("closing {}", self.name);  
    }  
}  
  
fn main() {  
    let _f = File::open("data.txt");  
} // _f goes out of scope → "closing data.txt"
```

- **Acquire** in open / new, **release** in drop
- Drop::drop runs **automatically** at end of scope
- Implement Drop::drop only if you require custom cleanup
- No manual close(), no forgotten cleanup, no leaks
- Similar to C++ destructors / [RAII](#) but guaranteed by ownership





Guarantee **exactly one** shared instance, reached through a **single access point** - e.g. one app config, one logger, one connection pool, one hardware resource.

Two promises: built **once**, then **shared** by all.

- **static INSTANCE:** OnceLock<Config> - the global slot, empty at start
- **get_or_init(...)** - **constructs** the Config once; later calls reuse it
- **returns** → **&'static Config** - callers get a **read-only** reference to the one instance

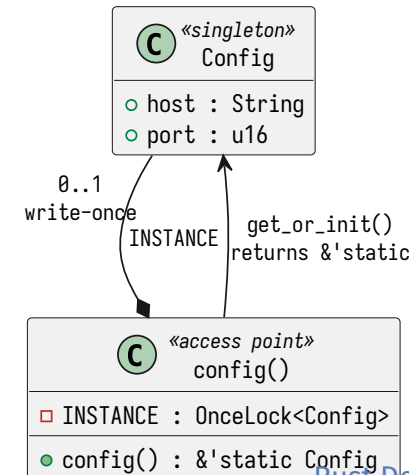
```
use std::sync::OnceLock;

struct Config {
    host: String,
    port: u16,
}

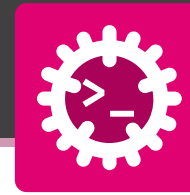
fn config() → &'static Config {
    // the single, write-once global slot
    static INSTANCE: OnceLock<Config> = OnceLock::new();

    INSTANCE.get_or_init(|| Config { // built on first call only
        host: "localhost".to_string(),
        port: 8080,
    })
}

fn main() {
    println!("{}", config().port); // first call → builds it
    println!("{}", config().host); // later call → same instance
}
```



Design Patterns — Singleton Pattern



Construct a complex object **step by step**

```
#[derive(Default)]
struct Server { host: String, port: u16, tls: bool }

struct ServerBuilder { server: Server } // the scaffold

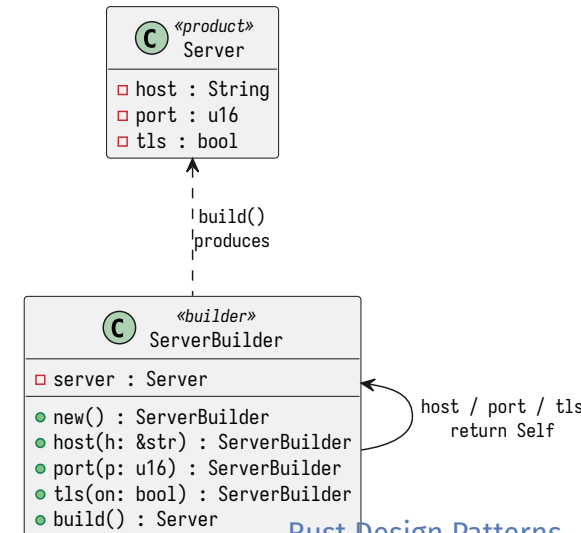
impl ServerBuilder {
    fn new() → Self { Self { server: Server::default() } }
    fn host(mut self, h: &str) → Self {
        self.server.host = h.into();
        self
    }
    fn port(mut self, p: u16) → Self { self.server.port = p; self }
    fn tls(mut self, on: bool) → Self { self.server.tls = on; self }
    fn build(self) → Server { self.server } // hand back product
}

fn main() {
    let s = ServerBuilder::new()
        .host("localhost") // set only what you need,
        .tls(true)         // in any order, by name
        .build();           // → Server { host, port: 0, tls:
true }
}
```

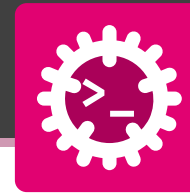
- each setting is optional, chosen by name
- build() produces the finished object



Avoids long positional constructors
new("localhost", 0, true) and a
maze of overloads for optional fields.



1. Each setter takes `self`, mutates, returns `self` → the chain.
2. `build()` consumes the builder and yields the product.
3. In `main`, `port` is never set → defaults to 0; order doesn't matter.



Make behaviour **interchangeable**: define a family of algorithms behind one trait, and let the caller pick or swap which one runs.

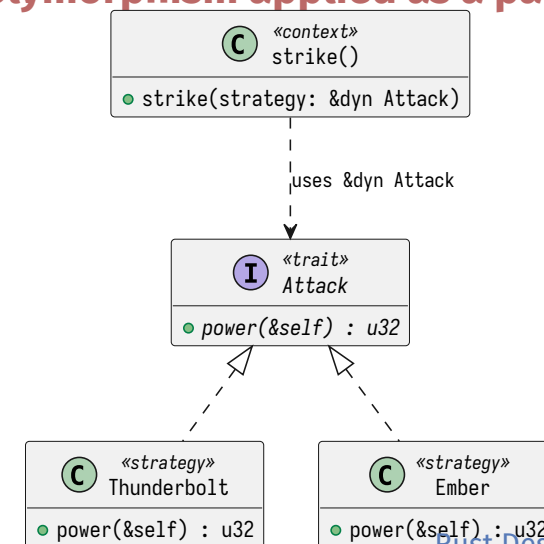
```
// the strategy interface
trait Attack {
    fn power(&self) -> u32;
}

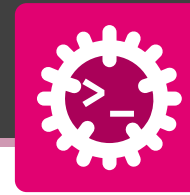
// concrete strategies
struct Thunderbolt;
impl Attack for Thunderbolt { fn power(&self) -> u32 { 90 } }
struct Ember;
impl Attack for Ember { fn power(&self) -> u32 { 40 } }

// the context: uses a strategy, doesn't know which one
// Accept any type implementing the Attack trait
fn strike(strategy: &dyn Attack) {
    println!("hit for {}", strategy.power());
}

fn main() {
    strike(&Thunderbolt); // hit for 90
    strike(&Ember);       // hit for 40 (swapped)
}
```

- Each algorithm is a type implementing a shared **trait** (Attack)
- Swap behaviour via:
 - &dyn Trait dynamic dispatch (run time)
 - <T: Trait> static dispatch (compile time)
- This is **polymorphism applied as a pattern**





Use the **Newtype** pattern so a user-id can never be confused with a product-id.

1. Define `UserId(u32)` and `ProductId(u32)` as tuple structs.
2. Write `fn load_user(id: UserId)` that prints the inner number.
3. Show that `load_user(ProductId(7))` and `load_user(7)` do **not** compile.



A one-field tuple struct is a **distinct type** at **zero runtime cost**.

**Solution**

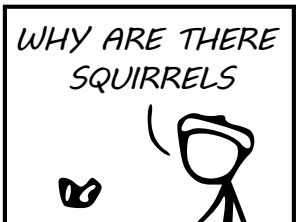
```
struct UserId(u32);
struct ProductId(u32);

fn load_user(id: UserId) {
    println!("loading user {}", id.0);
}

fn main() {
    load_user(UserId(7));    // ✓ correct type
    // load_user(ProductId(7)); // × different newtype
    // load_user(7);         // × plain u32, not a UserId
}
```

WHY DO WHALES JUMP
WHY ARE THERE MIRRORS ABOVE BEDS
WHY DO I SAY UH
WHY IS SEA SALT BETTER
WHY ARE THERE TREES IN THE MIDDLE OF FIELDS
WHY IS THERE NOT A POKEMON MMO
WHY IS THERE LAUGHING IN TV SHOWS
WHY ARE THERE DOORS ON THE FREEWAY
WHY ARE THERE SO MANY SVCHOST.EXE RUNNING
WHY AREN'T ANY COUNTRIES IN ANTARCTICA
WHY ARE THERE SCARY SOUNDS IN MINECRAFT
WHY IS THERE KICKING IN MY STOMACH
WHY ARE THERE TWO SLASHES AFTER HTTP
WHY ARE THERE CELEBRITIES
WHY DO SNAKES EXIST
WHY DO OYSTERS HAVE PEARLS
WHY ARE DUCKS CALLED DUCKS
WHY DO THEY CALL IT THE CLAP
WHY ARE KYLE AND CARTMAN FRIENDS
WHY IS THERE AN ARROW ON AANG'S HEAD
WHY ARE TEXT MESSAGES BLUE
WHY ARE THERE MUSTACHES ON CLOTHES
WHY WUBA LUBBA DUB DUB MEANING
WHY IS THERE A WHALE AND A POT FALLING
WHY ARE THERE SO MANY BIRDS IN SWISS
WHY IS THERE SO LITTLE RAIN IN WALLIS
WHY IS WALLIS WEATHER FORECAST ALWAYS WRONG
WHY ARE THERE MALE AND FEMALE BIKES

WHY ARE THERE BRIDESMAIDS
WHY DO DYING PEOPLE REACH UP
HOW FAST IS LIGHTSPEED
WHY ARE OLD KLINGONS DIFFERENT

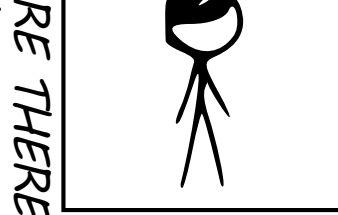


WHY ARE THERE TINY SPIDERS IN MY HOUSE
WHY DO SPIDERS COME INSIDE
WHY ARE THERE HUGE SPIDERS IN MY HOUSE
WHY ARE THERE LOTS OF SPIDERS IN MY HOUSE
WHY ARE THERE SPIDERS IN MY ROOM
WHY ARE THERE SO MANY SPIDERS IN MY ROOM
WHY DO SPYDER BITES ITCH

WHY ARE HATS SO EXPENSIVE
WHY IS THERE CAFFEINE IN MY SHAMPOO
WHY HAVE DINOSAURS NO FUR
WHY DO IGUANAS DIE
WHY AREN'T ECONOMISTS RICH
WHY DO AMERICANS CALL IT SOCCER
WHY ARE MY EARS RINGING
WHY IS 42 THE ANSWER TO EVERYTHING
WHY CAN'T NOBODY ELSE LIFT THORS HAMMER
WHY IS MARVIN ALWAYS SO SAD
WHY IS EARTH TILTED
WHY IS SPACE BLACK
WHY IS OUTER SPACE SO COLD
WHY ARE THERE PYRAMIDS ON THE MOON
WHY IS NASA SHUTTING DOWN
WHY ARE THERE FEMALE

WHY ARE THERE TINY SPIDERS IN MY HOUSE
WHY DO SPIDERS COME INSIDE
WHY ARE THERE HUGE SPIDERS IN MY HOUSE
WHY ARE THERE LOTS OF SPIDERS IN MY HOUSE
WHY ARE THERE SPIDERS IN MY ROOM
WHY ARE THERE SO MANY SPIDERS IN MY ROOM
WHY DO SPYDER BITES ITCH

WHY ARE TWINS HAVE DIFFERENT FINGERPRINTS
WHY ARE SWISS AFRAID OF DRAGONS
WHY IS THERE A SWARM OF ANTS
WHY IS THERE PILGRIM
WHY IS THERE LAVA
WHY AREN'T THERE GUNS IN HARRY POTTER
WHAT IS <https://xkcd.com/1256/>
WHY DO THEY SAY T-MINUS
WHY ARE OCEANS BECOMMING MORE ACIDIC



WHY IS THERE AN OWL IN MY BACKYARD
WHY IS THERE AN OWL OUTSIDE MY WINDOW
WHY IS THERE AN OWL ON THE DOLLAR BILL
WHY DO OWLS ATTACK PEOPLE
WHY ARE FPGA's EVERYWHERE
WHY ARE THERE HELICOPTERS CIRCLING MY HOUSE
WHY ARE THERE GODS
WHY ARE THERE TWO SPOCKS
WHY IS LI

WHY ARE MY BOOBS ITCHY
WHY ARE CIGARETTES LEGAL
WHY ARE THERE DUCKS IN MY POOL
WHY IS JESUS WHITE
WHY IS THERE LIQUID IN MY EAR
WHY DO Q TIPS FEEL GOOD
WHY DO PEOPLE DIE

QUESTIONS

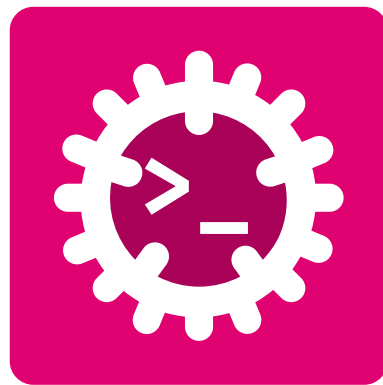
CAN BE ASKED BY ANYONE ANYTIME

WHY IS HTTPS CROSSED OUT
WHY IS THERE A LINE THROUGH
WHY IS THERE A RED LINE THROUGH HTTPS
WHY IS HTTPS IMPOR
WHY IS YKK ON ALL ZIPPERS

WHY ARE THERE SO MANY CROWS IN ROCHESTER
WHY IS TO BE OR NOT TO BE FL
WHY DO CHILDREN GET CANCER
WHY IS POSEIDON ANGRY WITH ODYSSEUS
WHY IS THERE ICE IN SPACE

WHY IS THERE AN OWL IN MY BACKYARD
WHY IS THERE AN OWL OUTSIDE MY WINDOW
WHY IS THERE AN OWL ON THE DOLLAR BILL
WHY DO OWLS ATTACK PEOPLE
WHY ARE FPGA's EVERYWHERE
WHY ARE THERE HELICOPTERS CIRCLING MY HOUSE
WHY ARE THERE GODS
WHY ARE THERE TWO SPOCKS

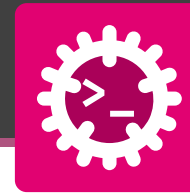
WHY ARE MY BOOBS ITCHY
WHY ARE CIGARETTES LEGAL
WHY ARE THERE DUCKS IN MY POOL
WHY IS JESUS WHITE
WHY IS THERE LIQUID IN MY EAR
WHY DO Q TIPS FEEL GOOD
WHY DO PEOPLE DIE



Glossary & Bibliography

Glossary & Bibliography

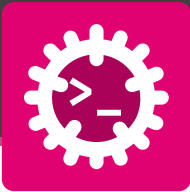
—



Programming

OOP – Object-Oriented Programming: Programming paradigm based on abstraction, encapsulation, modularity and hierarchy. [4](#)

RAII – Resource Acquisition Is Initialization: Resource management technique where acquisition and release are tied to object lifetime. [2](#), [14](#), [14](#), [20](#), [22](#), [22](#), [22](#)



Bibliography

[1] S. Klabnik, C. Nichols, and C. Krycho, “The Rust Programming Language.”

[2] J. Blandy, J. Orendorff, and L. F. s Tindall, *Programming Rust: Fast, Safe Systems Development*. O'Reilly Media, Incorporated, 2021.

[3] H. Collard, *Black Hat Rust*, vol. 1. Amazon, 2025.

[4] A. Shvets, “Refactoring Guru.” 2026.

[5] R. C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship*. Pearson Education, 2008.

[6] “SVG Repo - Free SVG Vectors and Icons.”